Guia de início da pesquisa — Onde revisar primeiro

26 de setembro de 2026 · Giorgio Nicolas de A. Bertino · CDIA — UFPB

> **Pergunta:** sinais da trajetória de um agente de IA indicam quais hunks de um patch estão errados melhor que regras simples, como sorteio ou tamanho do hunk?

Este guia leva do zero até os primeiros resultados. Nesta pasta estão também o `mini_projeto_01.py` (primeiro contato com os dados) e o `diario-de-pesquisa.md` (registro de cada sessão). A proposta para o professor está em `Documentos\Proposta-de-pesquisa-Onde-revisar-primeiro.pdf`.

1. Plano por fases

Quando	Fase	Entrega
28/09 – 11/10	Ambiente + mini-projeto 01	Tabela e gráfico comparando trajetórias que resolveram e que falharam
Até 11/10	Conversa com o professor	Levar a proposta e os primeiros números do mini-projeto
Out – nov	Base teórica	Ler os 6 papers da proposta e a área de defect prediction; resumo de 1 parágrafo por paper no diário
Nov – dez	Gabarito piloto (com orientador)	Definição de "hunk errado" testada em ~20 patches com os testes do SWE-bench
Jan – mar/2027	Primeiros resultados	Pistas extraídas por hunk, comparadas com as baselines; números que entram na candidatura do Tech Fellow

As datas a partir de novembro dependem do orientador. Ritmo sugerido: 6 a 10 horas por semana, sem tirar o foco do lançamento do Catenary.

2. Preparar o ambiente (uma vez)

No terminal, dentro da pasta `Documentos\Pesquisa-Onde-Revisar`:

```
python -m venv .venv
.venv\Scripts\activate
pip install datasets pandas matplotlib
```

O ambiente virtual (`.venv`) isola os pacotes da pesquisa do resto do computador. Sempre ative antes de trabalhar.

3. Mini-projeto 01: conhecer os dados

Dados: nebius/SWE-agent-trajectories no Hugging Face, com 80.036 trajetórias de um agente tentando resolver issues reais do GitHub (licença CC-BY-4.0, cite na publicação). Cada linha tem o patch gerado, a trajetória completa e se a tarefa foi resolvida. O script lê só 300 trajetórias em modo streaming, sem baixar os 1,1 GB.

1. **Célula 1–2:** carrega e imprime uma trajetória. Leia com calma: é o "vídeo" do agente trabalhando.
2. **Célula 3:** conta pistas por trajetória (passos, edições, leituras, buscas, testes, hunks). Confira na lista de comandos mais comuns se as categorias fazem sentido e ajuste.
3. **Célula 4–5:** compara medianas entre quem resolveu e quem falhou e salva um gráfico e um CSV.
4. **Célula 6:** perguntas para responder no diário, com números.

> O script não foi testado nesta máquina (sem rede para instalar os pacotes durante a preparação). Se algo quebrar, a célula 2 mostra o formato real dos dados; ajuste a partir dela. Isso faz parte do trabalho com dados reais.

Este mini-projeto é por **trajetória**, não por hunk. Ele serve para aprender o formato e já dá o que mostrar ao professor.

4. O que aprender, na ordem em que a pesquisa pede

Tema	Para quê	Onde
Python + pandas + Jupyter	Transformar trajetórias em tabelas	Kaggle Learn (Python, Pandas); livro *Python for Data Analysis* (versão gratuita online)
JSON e diffs	Ler trajetórias; quebrar patches em hunks (biblioteca `unidiff`)	Na prática, no mini-projeto
Docker básico	Rodar os testes do SWE-bench para montar o gabarito	Documentação oficial do Docker; README do SWE-bench
Estatística	Intervalo de confiança, comparar métodos, bootstrap	StatQuest (YouTube); disciplinas do CDIA
Métricas de ranking	Recall@k e recall por esforço ("revisando 20%, achei quanto?")	Papers de defect prediction
ML básico (depois)	Combinar pistas: regressão logística, random forest	Kaggle Learn (Intro to ML); scikit-learn

5. Leitura

Os 6 papers da proposta

AgentLens (2605.12925), TraceProbe (2607.06184), Strained Coherence (2606.07889), Early-Stage Review Effort (2601.00753), Vasconcelos et al. (2302.07248) e Majgaonkar et al. (2511.00197). Links na proposta. Como ler: resumo, figuras e conclusão primeiro; método só depois.

A área vizinha que o professor provavelmente vai citar

Defect prediction, em especial *just-in-time* (por commit) e *line-level* (por linha). Ela prevê onde há bug a partir do código e do histórico do git e usa métricas de esforço de revisão. Ponto de partida: Kamei et al., *A Large-Scale Empirical Study of Just-in-Time Quality Assurance*, IEEE TSE, 2013. A diferença da sua pesquisa: o sinal vem do **rastro do agente**, um tipo de dado que só passou a existir com agentes de IA.

6. Cuidados

- **Vazamento de dados:** nenhuma pista pode usar informação que só existe por causa do resultado dos testes finais (ex.: a mensagem de falha da avaliação). Senão o método "cola" do gabarito.
- **Baselines sempre:** todo resultado é comparado com sorteio e com "maior hunk primeiro".
- **Novidade:** a área publica toda semana. Refazer a busca de literatura com o orientador antes de investir no experimento.
- **Não prometer:** nem publicação, nem que é inédito. "Minhas buscas não encontraram" é a frase certa.
- **Licenças:** citar os datasets usados (CC-BY-4.0 exige atribuição).

7. Glossário

Termo	Significado
Diff / patch	A diferença entre o código antes e depois; o patch é o conjunto de mudanças que o agente entrega
Hunk	Cada bloco de mudança dentro do diff (começa com @@)
Trajetória	O registro passo a passo do agente: o que leu, editou, rodou
Gabarito (ground truth)	A resposta certa de quais hunks estão errados, usada para avaliar o palpite
Baseline	O competidor simples que o método precisa vencer (sorteio, tamanho)
Recall@k	Com que frequência o hunk errado está entre os k primeiros da fila
SWE-bench	Benchmark de bugs reais do GitHub com testes, usado para avaliar agentes

8. Próximas ações

- ☐ Preparar o ambiente (seção 2)
- ☐ Rodar o mini-projeto 01 e anotar os números no diário
- ☐ Escolher 2 ou 3 professores de Engenharia de Software ou IA na UFPB e pedir uma conversa
- ☐ Ler AgentLens e Vasconcelos et al. primeiro (os dois extremos da lacuna)